Simulação de Algoritmos de Substituição de Páginas

Avaliação do número de falhas de página com OPT, LRU e LRU aproximado

Trabalho 2 de Sistemas Operacionais

Rafaela Pacheco Nunes

Outubro de 2026

Sumário

1. Introdução **3**

1.1 Enunciado . . . 3

1.2 Interpretação adotada . . . 3

2. Fundamentação teórica **3**

2.1 Endereços e páginas . . . 3

2.2 Algoritmo ótimo (OPT) . . . 4

2.3 LRU (Least Recently Used) . . . 4

2.4 LRU aproximado com bits de referência adicionais (aging) . . . 4

3. Implementação **4**

3.1 Modelo de simulação . . . 4

3.2 OPT . . . 5

3.3 LRU exato . . . 5

3.4 LRU aproximado . . . 5

3.5 Programa principal e gráficos . . . 5

4. Validação **6**

4.1 Testes automatizados . . . 6

4.2 Verificação cruzada com implementações ingênuas . . . 6

4.3 Integridade dos traces . . . 6

5. Metodologia experimental **6**

5.1 Traces . . . 6

5.2 Parâmetros . . . 6

5.3 Ambiente e tempo de execução . . . 7

6. Escolha de N e T: análise de sensibilidade **7**

7. Resultados **9**

7.1 bzip . . . 9

7.2 gcc . . . 10

7.3 sixpack . . . 11

7.4 swim . . . 12

7.5 bigone . . . 13

8. Discussão **14**

8.1 Efeito do número de frames . . . 14

8.2 OPT como referência . . . 14

8.3 Qualidade da aproximação do LRU . . . 14

9. Conclusão **14**

10. Atendimento aos requisitos **15**

Referências **15**

1. Introdução

Em um sistema com memória virtual paginada, apenas parte das páginas de um processo fica na memória física, dividida em quadros (*frames*). Quando o processo acessa uma página que não está em memória, ocorre uma **falha de página**: o sistema operacional precisa buscar a página no disco e, se não houver frame livre, escolher uma página vítima para sair. Como o acesso ao disco é ordens de grandeza mais lento que o acesso à memória, a política usada para escolher a vítima tem impacto direto no desempenho.

Este trabalho avalia esse impacto por simulação. Foi desenvolvido um programa que lê *traces* reais de acesso à memória e conta o número de falhas de página produzido por três políticas de substituição, variando a quantidade de frames disponíveis. Para o LRU aproximado, também foram avaliados diferentes tamanhos do registrador de histórico e diferentes intervalos de envelhecimento.

1.1 Enunciado

O enunciado pede para avaliar o impacto da política de substituição no número de falhas de página, de acordo com os algoritmos previstos na seção de gerenciamento de memória, comparando três abordagens e diferentes números de frames escolhidos pelo aluno. Para o LRU aproximado, o número de bits de histórico deve poder ser definido pelo programa, assim como o intervalo em que os bits são deslocados. Os traces têm o formato `31348900 W` (endereço em hexadecimal e tipo de acesso), os endereços são de uma arquitetura de 32 bits e as páginas têm 4096 bytes.

1.2 Interpretação adotada

- **Três abordagens**: foram interpretadas como três políticas de substituição. O trabalho de referência pede OPT e LRU aproximado; a terceira escolhida foi o **LRU exato**, por ser a política que o LRU aproximado tenta imitar, o que permite medir a qualidade da aproximação.
- **Intervalo de envelhecimento**: os traces registram apenas acessos à memória, não instruções. Por isso o intervalo é medido em número de acessos, a mesma unidade usada no trabalho de referência.
- **Falhas compulsórias**: o primeiro acesso a cada página também é contado como falha, pois a memória começa vazia.

2. Fundamentação teórica

2.1 Endereços e páginas

Com páginas de $4096 = 2^{12}$ bytes, um endereço virtual de 32 bits se divide em um deslocamento de 12 bits (posição dentro da página) e um número de página de 20 bits. O número da página é obtido deslocando o endereço 12 bits para a direita:

```
0x31348900 >> 12 = 0x31348
```

O deslocamento e o tipo de acesso (leitura ou escrita) não alteram o número de falhas, então a simulação trabalha apenas com a sequência de números de página.

2.2 Algoritmo ótimo (OPT)

Proposto por Belady, o OPT expulsa a página cujo próximo uso está mais distante no futuro, ou que nunca mais será usada. Nenhum outro algoritmo produz menos falhas para a mesma sequência de acessos. Não pode ser implementado em um sistema real, pois exige conhecer os acessos futuros, mas é viável na simulação porque o trace inteiro está disponível. Serve como **limite inferior** para avaliar as outras políticas.

2.3 LRU (*Least Recently Used*)

O LRU expulsa a página que está há mais tempo sem ser usada. Ele se baseia na localidade temporal: páginas usadas recentemente tendem a ser usadas de novo em breve. Implementá-lo de forma exata em hardware exige atualizar uma ordenação ou um carimbo de tempo a cada acesso à memória, o que é caro demais. Por isso, na prática, os sistemas usam aproximações.

2.4 LRU aproximado com bits de referência adicionais (*aging*)

Cada página em memória tem um **bit de referência R**, que o hardware coloca em 1 a cada acesso, e um **registrador de histórico de N bits**, mantido pelo sistema operacional. Em intervalos regulares de **T acessos**, o sistema operacional percorre as páginas em memória e faz:

```
historico = (R << (N - 1)) | (historico >> 1)
R = 0
```

Assim, o registrador guarda se a página foi usada em cada um dos últimos N intervalos, com os intervalos mais recentes nos bits de maior peso. Interpretado como número, quanto menor o valor, há mais tempo a página não é usada, e é ela a escolhida como vítima. O produto N × T define a "memória" do algoritmo: páginas sem uso nos últimos N × T acessos ficam com histórico zerado e se tornam indistinguíveis entre si.

3. Implementação

O simulador foi escrito em Python 3, sem dependências além do matplotlib (gráficos) e do pytest (testes). A estrutura do projeto é:

```
simulacao-algoritmos-so/
├── src/
│   ├── leitor_trace.py   leitura do trace e conversão de endereço em página
│   ├── algoritmos.py     OPT, LRU exato e LRU aproximado
│   ├── main.py           simulador: executa os algoritmos e grava o CSV
│   └── graficos.py       gera os gráficos a partir do CSV
├── tests/
│   └── test_algoritmos.py
├── resultados/           CSVs, logs de execução e gráficos
├── traces/               traces do moodle (fora do controle de versão)
├── README.md
├── pytest.ini
└── requirements.txt
```

3.1 Modelo de simulação

Cada trace é lido uma única vez e convertido em uma lista de números de página, reaproveitada por todos os algoritmos. Cada algoritmo recebe essa lista e o número de frames F, e devolve o número de falhas. A memória começa vazia; a cada acesso, se a página está em memória há um acerto, senão há uma falha. Na falha, a página ocupa um frame livre se houver; caso contrário, a política escolhe a vítima.

3.2 OPT

Uma implementação ingênua, que procura no restante do trace o próximo uso de cada página a cada falha, seria inviável para traces de milhões de acessos. A implementação adotada tem duas etapas:

- uma passada de trás para frente calcula, para cada posição *i*, a posição do próximo acesso à mesma página (`proximo_uso[i]`), usando infinito quando ela não aparece mais;
- as páginas em memória ficam em um *max-heap* ordenado pelo próximo uso. Quando uma página é acessada de novo, uma nova entrada é inserida e a antiga fica desatualizada; ela é descartada quando chega ao topo (remoção preguiçosa).

Com isso, a vítima é encontrada em tempo O(log n), e a simulação completa custa O(n log n).

3.3 LRU exato

Usa um `OrderedDict` com as páginas da menos para a mais recentemente usada. Em um acerto, a página é movida para o fim; em uma falha com a memória cheia, sai a primeira. Todas as operações são O(1), então a simulação custa O(n).

3.4 LRU aproximado

O registrador de histórico de cada página fica em um dicionário, e o conjunto das páginas com R = 1 em um `set`. A cada T acessos, todos os históricos são deslocados, as páginas do conjunto recebem o bit mais significativo e o conjunto é esvaziado. As decisões de projeto foram:

- **Escolha da vítima**: a página com menor valor do par (R, histórico). O bit R é comparado primeiro porque indica uso desde o último deslocamento, informação mais recente que qualquer bit do histórico.
- **Desempate**: entre páginas com o mesmo par, sai a carregada há mais tempo (critério FIFO).
- **Página recém-carregada**: entra com R = 1 e histórico zerado.
- **Parâmetros**: N e T são definidos pelas opções `--bits` e `--intervalo`. Os valores padrão, **N = 32** e **T = 100**, foram escolhidos pela análise de sensibilidade da seção 6.

O custo é O(n × F / T) para os deslocamentos mais O(F) por falha para escolher a vítima.

3.5 Programa principal e gráficos

O `main.py` recebe os traces e as listas de frames, bits e intervalos, executa OPT e LRU uma vez por quantidade de frames e o LRU aproximado para cada combinação de N e T. Os resultados são mostrados no terminal, com o tempo de cada execução, e gravados em CSV. O `graficos.py` gera, a partir do CSV, o gráfico de falhas por número de frames de cada trace e, com a opção `--sensibilidade`, os gráficos da análise de N e T.

```
python src/main.py traces/*.trace.txt
python src/main.py traces/*.trace.txt --frames 8 32 128 --bits 2 4 8 16 32 \
    --intervalo 10 100 1000 10000 --saida resultados/sensibilidade.csv
python src/graficos.py resultados/resultados.csv
python src/graficos.py resultados/sensibilidade.csv --sensibilidade
```

4. Validação

4.1 Testes automatizados

O projeto tem 17 testes automatizados (`python -m pytest`), todos aprovados. Eles combinam um caso com resultado conhecido e propriedades que qualquer implementação correta deve respeitar:

- a sequência de referência clássica do livro de Silberschatz, `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1`, que com 3 frames deve resultar em **9 falhas no OPT e 12 no LRU**;
- o LRU aproximado com T = 1 e bits suficientes deve se comportar exatamente como o LRU;
- o OPT nunca pode ter mais falhas que os outros algoritmos;
- com memória suficiente para todas as páginas, só podem ocorrer as falhas compulsórias;
- com um único frame, toda troca de página entre acessos consecutivos é uma falha;
- a conversão de endereço em número de página e a leitura do trace.

4.2 Verificação cruzada com implementações ingênuas

Como verificação adicional, os três algoritmos foram comparados com implementações ingênuas, escritas de forma independente e fáceis de conferir manualmente (o OPT procurando o próximo uso no trace a cada falha, o LRU guardando o instante do último acesso e o LRU aproximado com um único registrador contendo R e o histórico). A comparação usou os 3000 primeiros acessos dos traces gcc, swim e bzip, com 3, 8 e 20 frames e as combinações (N, T) = (2, 10), (8, 100) e (32, 100). Todos os resultados coincidiram.

4.3 Integridade dos traces

Todas as 8 milhões de linhas dos cinco traces seguem o formato esperado (oito dígitos hexadecimais, um espaço e R ou W), sem nenhuma linha malformada.

5. Metodologia experimental

5.1 Traces

Foram usados os cinco traces disponibilizados no moodle. A tabela mostra o número de acessos e de páginas distintas de cada um; este último é o mínimo de falhas possível, pois cada página distinta gera ao menos uma falha compulsória.

Trace	Acessos	Páginas distintas
bzip	1.000.000	317
gcc	1.000.000	2.852
sixpack	1.000.000	3.890
swim	1.000.000	2.543
bigone	4.000.000	8.254

Tabela 1: traces utilizados.

5.2 Parâmetros

- **Número de frames**: 4, 8, 12, 16, 20, 24, 28 e 32 (passo de 4, como sugerido no trabalho de referência), mais 64, 128 e 256, para observar o comportamento com memória mais folgada.

- **LRU aproximado, experimento principal**: N = 32 bits e T = 100 acessos.
- **LRU aproximado, análise de sensibilidade**: N em {2, 4, 8, 16, 32} e T em {10, 100, 1000, 10000}, com 8, 32 e 128 frames, em todos os traces.

5.3 Ambiente e tempo de execução

Os experimentos foram executados em Python 3.13 no Windows 11. No experimento principal (5 traces e 11 quantidades de frames), o tempo total de simulação foi de 73 s para o OPT, 10 s para o LRU e 56 s para o LRU aproximado. Os tempos de cada execução estão em `resultados/execucao.log`.

6. Escolha de N e T: análise de sensibilidade

Antes do experimento principal, o LRU aproximado foi executado com todas as combinações de N e T listadas na seção 5.2. Para cada combinação, calculou-se a razão entre as falhas do LRU aproximado e as do LRU exato em cada trace e quantidade de frames, e a média geométrica dessas razões. A Tabela 2 mostra quanto o LRU aproximado ficou acima do LRU exato em média (valores negativos indicam menos falhas).

N \ T	10	100	1000	10000
2	+19,3%	+9,8%	+8,6%	+13,8%
4	+16,7%	+7,5%	+6,6%	+13,2%
8	+12,4%	+5,3%	+5,9%	+12,6%
16	+8,8%	+2,8%	+5,6%	+12,6%
32	+7,0%	+1,1%	+5,4%	+12,6%

Tabela 2: diferença média de falhas do LRU aproximado em relação ao LRU exato.

N	T	8 frames	32 frames	128 frames	Geral
32	100	+0,2%	-0,9%	+4,0%	+1,1%
16	100	+0,3%	-0,7%	+9,0%	+2,8%
8	100	+0,3%	+0,1%	+16,3%	+5,3%
32	1000	+10,3%	+6,9%	-0,8%	+5,4%
16	1000	+10,3%	+7,1%	-0,2%	+5,6%

Tabela 3: as cinco melhores combinações, por quantidade de frames.

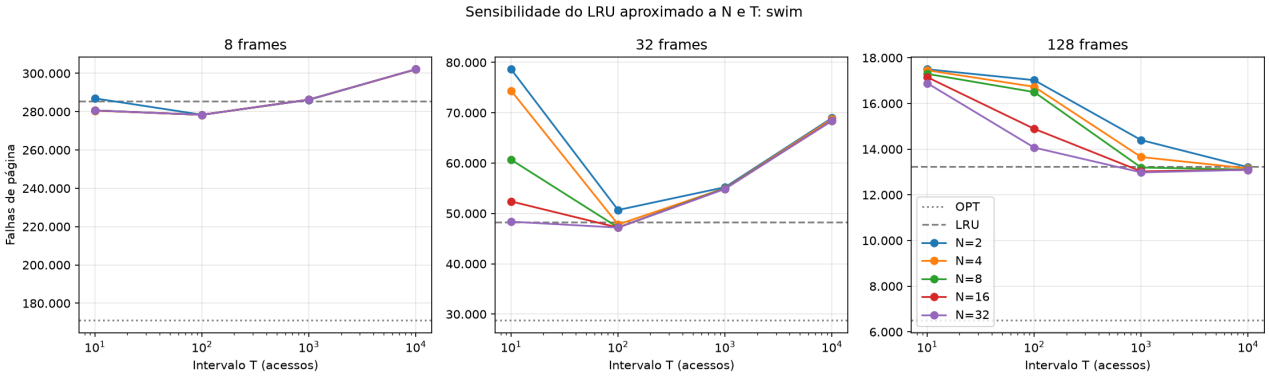


Figura 1: falhas do LRU aproximado em função de T no trace swim. As linhas tracejada e pontilhada indicam o LRU exato e o OPT.

Principais observações:

- **T grande demais é ruim na média.** Com T = 10000 e N a partir de 8 bits, o LRU aproximado teve o pior resultado médio, entre 12,6% e 13,8% acima do LRU exato, e ficou mais de 17% acima com 8 e 32 frames. Num intervalo tão longo, quase todas as páginas em memória são acessadas e ficam com R = 1; a escolha da vítima perde a noção de ordem e degenera para FIFO. Aumentar N não ajuda, porque a informação perdida está dentro do intervalo. Só com 128 frames a janela longa compensa (entre 1,5% e 4,1% acima do LRU exato).
- **T pequeno demais exige muitos bits.** Com T = 10, a janela N × T é curta, muitas páginas ficam com histórico zerado e empatam. Mais bits compensam em parte.
- **Mais bits quase sempre ajudam.** Na média, o resultado melhora ou empata a cada aumento de N, para qualquer T. Caso a caso, em 12 das 240 comparações entre valores vizinhos de N houve piora, sempre pequena (no máximo 1,2%). O ganho é maior quando a janela N × T é curta em relação ao tempo em que as páginas ficam sem uso, o que ocorre com muitos frames (128).
- **N = 32 e T = 100 teve o melhor resultado geral**, +1,1% em relação ao LRU exato. Além disso, 32 bits cabem em uma palavra de máquina de uma arquitetura de 32 bits, o que torna a escolha realista. Esses foram os valores adotados.

7. Resultados

Esta seção apresenta o número de falhas de página de cada trace para os três algoritmos, com o LRU aproximado configurado com N = 32 e T = 100. A última coluna de cada tabela compara o LRU aproximado com o LRU exato. Os gráficos usam escala logarítmica no eixo dos frames, para que valores pequenos e grandes fiquem legíveis juntos.

7.1 bzip

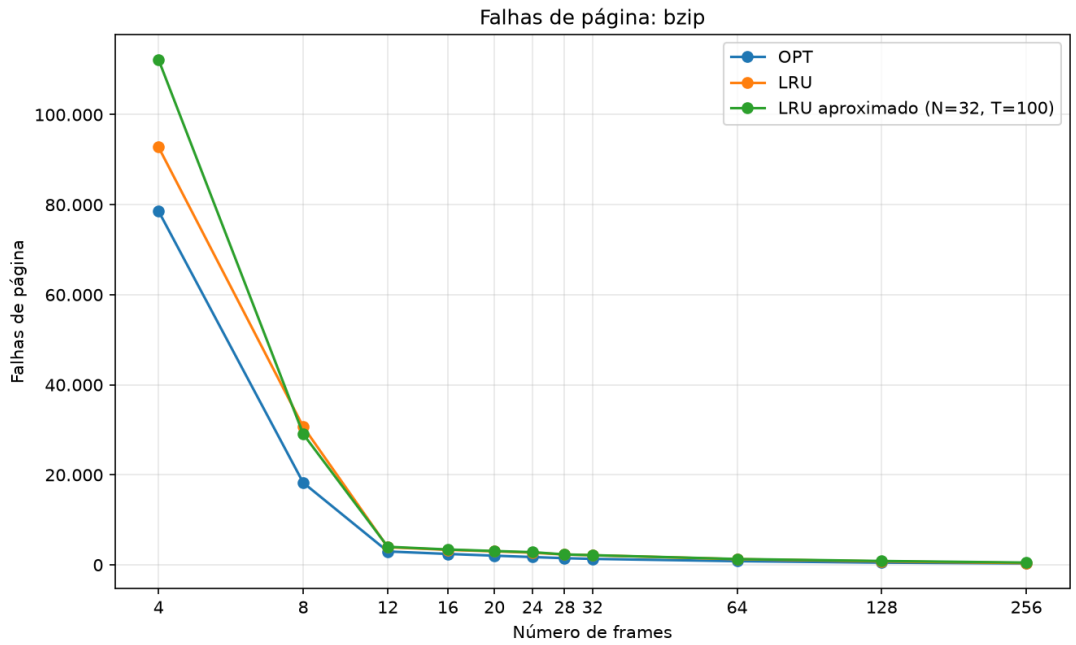


Figura 2: falhas de página no trace bzip.

Frames	OPT	LRU	LRU aproximado	Aprox. vs LRU
4	78.562	92.770	112.196	+20,9%
8	18.251	30.691	29.057	-5,3%
12	2.978	3.907	4.015	+2,8%
16	2.427	3.344	3.417	+2,2%
20	2.043	3.001	3.073	+2,4%
24	1.735	2.702	2.822	+4,4%
28	1.488	2.301	2.323	+1,0%
32	1.330	2.133	2.156	+1,1%
64	821	1.264	1.301	+2,9%
128	497	771	859	+11,4%
256	317	397	508	+28,0%

Tabela 4: falhas de página no trace bzip (1.000.000 acessos, 317 páginas distintas).

7.2 gcc

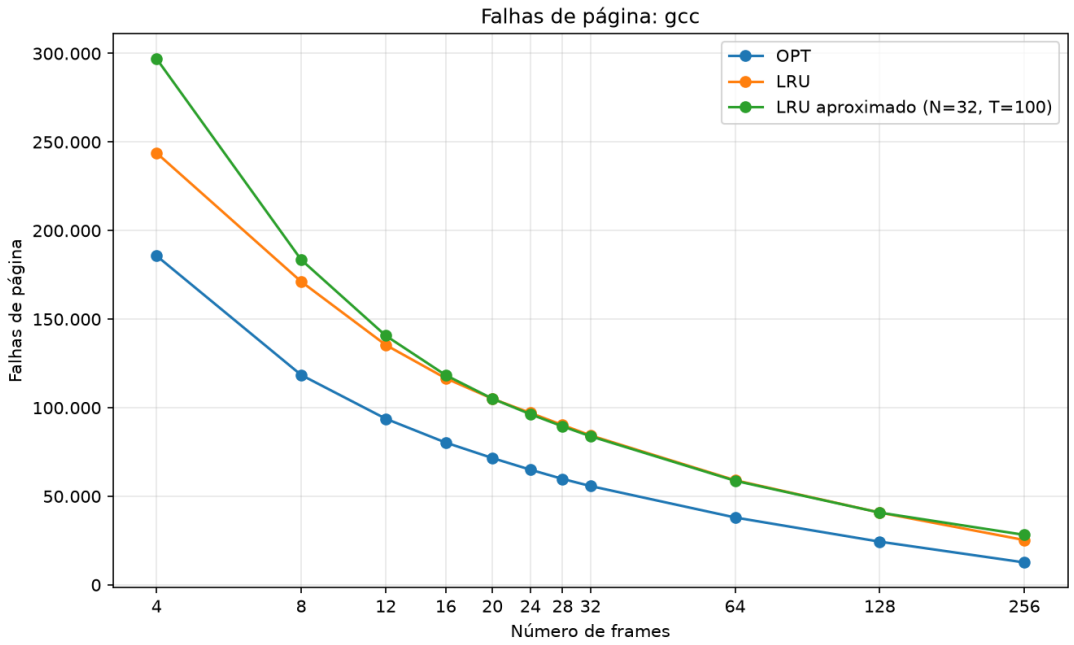


Figura 3: falhas de página no trace gcc.

Frames	OPT	LRU	LRU aproximado	Aprox. vs LRU
4	185.754	243.809	297.010	+21,8%
8	118.480	171.186	183.513	+7,2%
12	93.781	135.421	140.662	+3,9%
16	80.307	116.604	118.344	+1,5%
20	71.602	105.097	105.139	+0,0%
24	65.013	97.076	96.344	-0,8%
28	59.807	90.243	89.465	-0,9%
32	55.802	84.401	83.901	-0,6%
64	38.050	59.089	58.772	-0,5%
128	24.391	40.821	40.793	-0,1%
256	12.667	25.308	28.241	+11,6%

Tabela 5: falhas de página no trace gcc (1.000.000 acessos, 2.852 páginas distintas).

7.3 sixpack

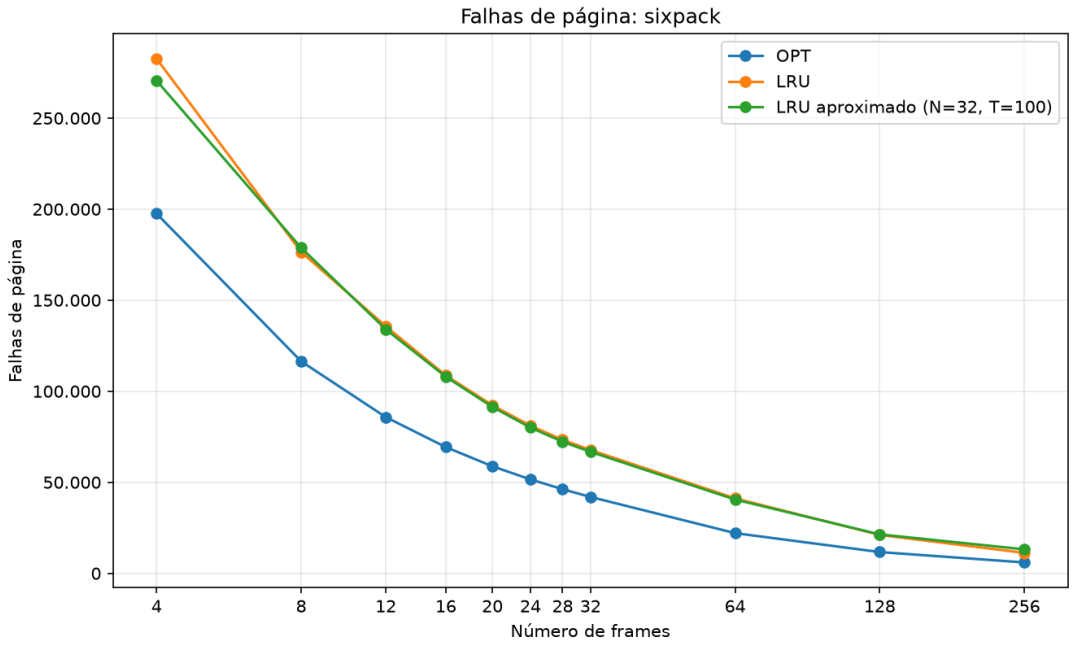


Figura 4: falhas de página no trace sixpack.

Frames	OPT	LRU	LRU aproximado	Aprox. vs LRU
4	197.551	282.620	270.605	-4,3%
8	116.407	176.496	178.765	+1,3%
12	85.779	135.644	134.006	-1,2%
16	69.325	108.682	107.971	-0,7%
20	58.809	92.319	91.375	-1,0%
24	51.576	81.089	80.053	-1,3%
28	46.203	73.341	72.297	-1,4%
32	41.955	67.747	66.744	-1,5%
64	22.083	41.186	40.529	-1,6%
128	11.690	21.090	21.404	+1,5%
256	5.981	11.240	13.169	+17,2%

Tabela 6: falhas de página no trace sixpack (1.000.000 acessos, 3.890 páginas distintas).

7.4 swim

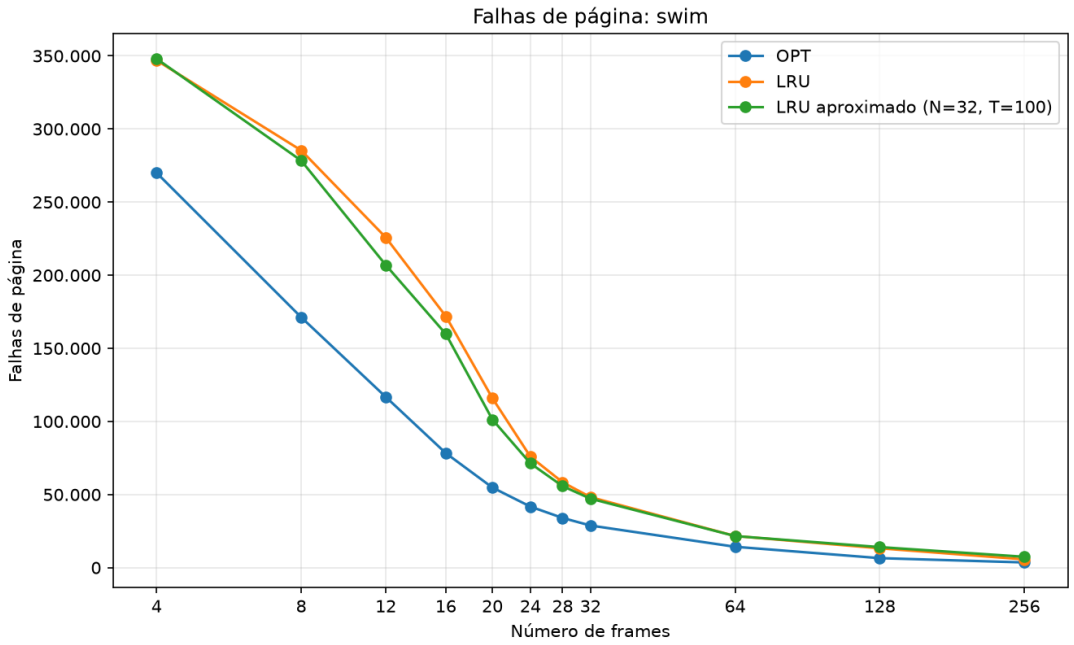


Figura 5: falhas de página no trace swim.

Frames	OPT	LRU	LRU aproximado	Aprox. vs LRU
4	270.046	346.936	348.046	+0,3%
8	171.244	285.375	278.376	-2,5%
12	116.727	225.819	206.940	-8,4%
16	78.312	171.961	159.820	-7,1%
20	54.799	115.939	101.339	-12,6%
24	41.751	75.882	71.312	-6,0%
28	34.002	58.425	55.755	-4,6%
32	28.826	48.254	47.195	-2,2%
64	14.289	21.656	21.617	-0,2%
128	6.518	13.250	14.068	+6,2%
256	3.572	5.673	7.438	+31,1%

Tabela 7: falhas de página no trace swim (1.000.000 acessos, 2.543 páginas distintas).

7.5 bigone

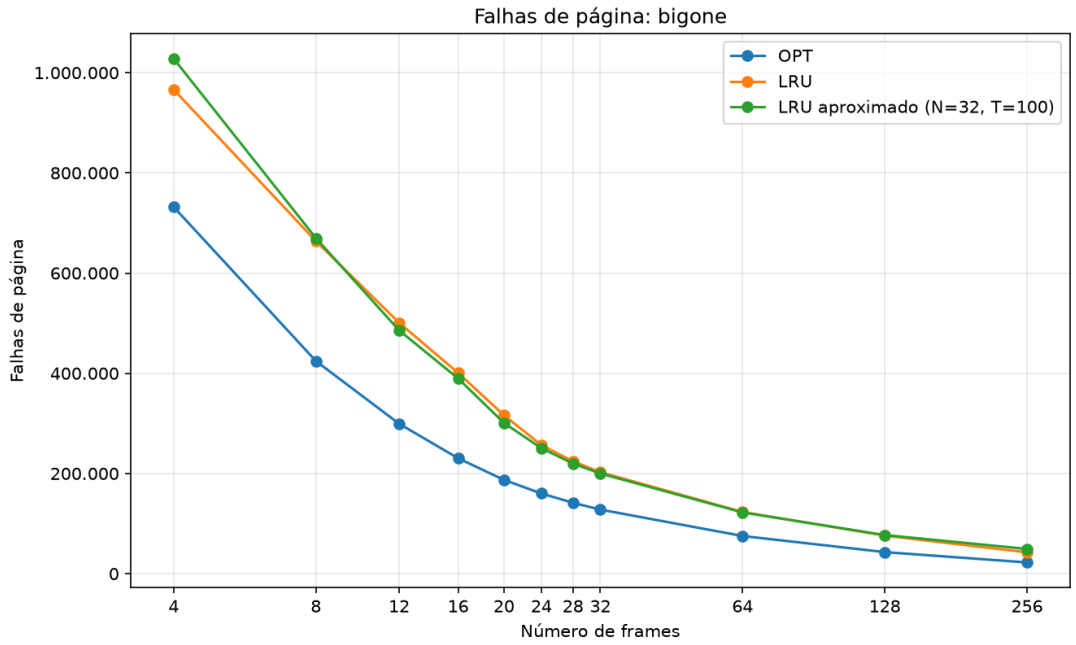


Figura 6: falhas de página no trace bigone.

Frames	OPT	LRU	LRU aproximado	Aprox. vs LRU
4	731.913	966.135	1.027.857	+6,4%
8	424.381	663.748	669.711	+0,9%
12	299.264	500.791	485.623	-3,0%
16	230.369	400.591	389.552	-2,8%
20	187.250	316.355	300.925	-4,9%
24	160.072	256.748	250.530	-2,4%
28	141.497	224.309	219.839	-2,0%
32	127.910	202.533	199.994	-1,3%
64	75.228	123.192	122.216	-0,8%
128	43.058	75.925	77.118	+1,6%
256	22.434	42.600	49.332	+15,8%

Tabela 8: falhas de página no trace bigone (4.000.000 acessos, 8.254 páginas distintas).

8. Discussão

8.1 Efeito do número de frames

Em todos os traces e algoritmos, o número de falhas diminui à medida que o número de frames aumenta. Não houve anomalia de Belady: OPT e LRU são algoritmos de pilha, imunes a ela, e o LRU aproximado, que não tem essa garantia, também não a apresentou nos valores testados.

O formato das curvas revela a localidade de cada programa. O bzip tem localidade muito forte: as falhas caem de forma abrupta entre 8 e 12 frames e depois a curva fica quase plana, indicando que seu conjunto de trabalho cabe em cerca de 12 frames. Com 256 frames o OPT chega a 317 falhas, exatamente o número de páginas distintas do trace, ou seja, apenas falhas compulsórias. Os demais traces têm conjuntos de trabalho maiores e quedas mais graduais; o swim, por exemplo, cai rapidamente entre 12 e 32 frames.

8.2 OPT como referência

O OPT teve o menor número de falhas em todos os casos. O LRU teve entre 18% e 120% mais falhas que o OPT, na maior parte dos casos entre 40% e 90%. Essa distância representa o ganho que seria possível com conhecimento do futuro, e mostra que mesmo uma boa política baseada no passado fica longe do ótimo.

8.3 Qualidade da aproximação do LRU

Na faixa intermediária, de 8 a 128 frames, o LRU aproximado ficou muito próximo do LRU exato e, em vários casos, teve até menos falhas, como no sixpack, no swim e no bigone. Isso acontece porque o registrador de histórico também captura a frequência de uso nos últimos intervalos, e não apenas o instante do último acesso, e o LRU exato não é ótimo: em alguns padrões de acesso, considerar a frequência ajuda.

Nas pontas, o LRU aproximado perde para o LRU exato. Com 4 frames, quase todas as páginas em memória são acessadas dentro do mesmo intervalo de 100 acessos, ficam com R = 1 e a escolha vira praticamente FIFO. Com 256 frames, muitas páginas ficam sem uso por mais que a janela de 3200 acessos, o histórico zera e elas empatam. Nos dois casos falta informação para distinguir as páginas, e o resultado depende do critério de desempate.

9. Conclusão

O simulador desenvolvido permitiu comparar as três políticas em cinco traces reais e onze quantidades de frames. Os resultados confirmam a teoria: o OPT é sempre o melhor e serve de limite inferior, o número de falhas cai com mais frames, e a forma dessa queda depende da localidade de cada programa.

O LRU aproximado com bits de referência mostrou ser uma alternativa viável ao LRU exato. Com parâmetros bem escolhidos, ficou em média a apenas 1,1% do LRU exato na análise de sensibilidade, com custo de implementação muito menor, já que só exige um bit de referência por página no hardware e um deslocamento periódico feito pelo sistema operacional. A análise de sensibilidade mostrou que N e T precisam ser escolhidos em conjunto: T controla a resolução temporal e N × T a duração da memória do algoritmo. Para os traces avaliados, N = 32 e T = 100 deram o melhor equilíbrio.

10. Atendimento aos requisitos

Exigência do enunciado	Como foi atendida
Avaliar o impacto da política de substituição no número de falhas	Simulação de três políticas em cinco traces (seções 7 e 8)
Comparar três abordagens	OPT, LRU exato e LRU aproximado (seções 2 e 3)
Diferentes números de frames, escolhidos pelo aluno	4 a 32 de 4 em 4, mais 64, 128 e 256
Programa que realiza a avaliação	src/main.py, com saída em tabela e CSV
Número de bits do LRU aproximado definido pelo programa	Opção --bits (padrão 32)
Intervalo de deslocamento dos bits	Opção --intervalo (padrão 100 acessos)
Usar os traces disponibilizados	bzip, gcc, sixpack, swim e bigone
Formato do trace e endereços de 32 bits	src/leitor_trace.py, com testes
Páginas de 4096 bytes	Número da página = endereço >> 12

Tabela 9: requisitos do enunciado e onde são atendidos.

Referências

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. 9. ed. Rio de Janeiro: LTC, 2015.

TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson, 2016.

BELADY, L. A. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, v. 5, n. 2, p. 78-101, 1966.